Creating a Distributed System Middle-ware Implementing and Optimizing X-Search

Shreya Bhatta
University of Texas at Arlington
shreya.bhatta@mavs.uta.edu

Alexandru Iulian Orhean
Illinois Institute of Technology
aorhean@hawk.iit.edu

Ioan Raicu
Illinois Institute of Technology
iraicu@cs.iit.edu

***Abstract*—With the escalation in use of distributed file systems for large-scale storage, locating and retrieving data has become progressively challenging. To facilitate information retrieval in existing parallel and distributed filesystems, a scalable solution, X-search, was proposed. In this paper, we discuss the implementation of a multi-node architecture that facilitates X-Search and provides inter-node communication while transferring complex messages in a distributed system.**

***Index Terms*—distributed, middle-ware, architecture**

I. Introduction

Implementing a middleware for distributed system facilitating information retrieval encompasses creating a communication model where messages can be interchanged between multiple nodes simultaneously. We were looking to create an architecture that handles multiple communication at once and provides a good latency. In this paper, we will discuss the design and structure of the middleware in section II and its implementation in section III. In section IV we discuss the performance of the middleware and in section V we go over some related information retrieval architecture. Finally, we conclude the paper in section VI with a conclusion and potential future work.

II. Design and structure

In order to understand the middle-ware, we need to understand its structure and component. One of the main goals of this project was to create a communication model that connects multiple nodes and facilitates sending messages and instructions from one node to another. For the inter-node communication of the middle-ware, we used socket. Socket is a communication channel which creates endpoint for communication and returns a file descriptor that refers to that endpoint. We decided to go with TCP/IP for the protocol. TCP/IP provides end-to-end communication that identifies how data should be broken into packets, addressed, transmitted, routed and received at the destination. Unlike UDP, another very commonly used protocol, TCP/IP facilitates a built-in system which checks for error and guarantees the data are sent in order.

We then had to come up with a model optimal for sending and receiving instructions. We were looking for high indexing performance and because the indexes are stored on the same nodes with storage nodes, we decided on a leader-worker model where workers do the indexing locally. Figure 1 shows the architecture of the middle-ware where leader provides instructions to each individual workers to do the indexing and the workers upon receiving the instruction from leader perform the indexing and then send the result if the indexing failed or succeeded back to the leader. A leader in this case, acts like a moderator. Whenever, there is a query, the same model applies, the leader sends the query to the workers and receives the results from all the workers and presents it to the user. As presented in the diagram, the worker nodes do not have any form of communication between them but they all individually have connection with the leader where they can send and receive messages.

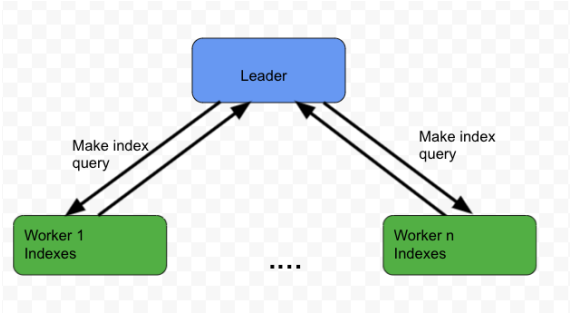

Fig. 1. Leader-Worker Model

III. Implementation

Because communication impacts the performance of search and indexing directly, we wanted the middle-ware capable of handling multiple connections while being able to send and receive messages from multiple file descriptors in a non-blocking manner. And when we send or receive a packet from a file descriptor, we wanted that to have very little overhead in latency.

A. Handling multiple connections

To handle and monitor multiple connections in the system we looked at few different options. Our initial goal was to create a multithreaded non-blocking system. We wanted our event processing to be concurrent with the dispatching scheme where a thread dispatches events to all the worker threads in the thread pool. But after a few trials and errors and complex debugging with non-blocking multithreaded system, because of the stretch of time, we decided to go with an easier route

by using a single-threaded non-blocking IO. In this system, events are pulled from the buffer and processed in sequence by the single execution unit [1]. Although this has a bottleneck in the event loop and CPU-bound tasks might degrade the performance, it has the advantage of a light context switch. Implementing a better connection model is definitely possible and will be one of the potential future works. We decided to go with select() for polling of events and handling connections. With select() a server/leader can monitor many clients/workers in a single process. Select() contains a list of file descriptors that let the leader know when there is data available for reading or writing from the workers. Select() also has a ”timeout” value which lets us regain control even when there has been no activity [2].

B. Sending/Receiving complex messages

When there is a request for query from the leader and the worker has multiple results that need to be sent back to the leader, there comes a need for a buffer that can store large results for leaders to view. That's when we consider two approaches optimal for sending and receiving complex messages, google protocol buffer and NFS.

a) Google Protocol Buffer: A protocol buffer is a language-neutral, platform-neutral open-source library used to serialize structured data [3]. It supports generated code in C objective, C++, Dart, Go, and Java. It is used to store and interchange complex structure data which makes it a very promising approach to send and receive data in the middleware for X-Search. Files in google protocol buffer can be up to 10 times smaller than that of XML and serialization in it can be as much as 100 times faster compared to XML serialization [4]. Data in the protocol buffer is transmitted in a binary format. Although binary is not as human-readable as a text file, this has its advantages. Because binary files take less space and bandwidth compared to text files, the speed of transmission is much faster and because the data is compressed, there is less CPU usage [5]. The implementation of protocol buffer is quite simple too. The literals that need to be sent are declared in a .proto file and upon running the protoc compiler, two files will be generated in the language that was specified which contain the getters and setters needed to access the attributes, as well as methods for serializing and parsing [4].

b) NFS: Another approach to send complex messages to the leader was with the implementation of a Network file system(NFS). Network file system is a protocol that allows a client node to access files that is local to the server node over a network. In our implementation, the leader runs nfsd to make a directory available and exports the directory that it wants to be shared and the workers request access to the directory using a mount command. After receiving the access, the workers then become able to read and write on the files in the directory as if they were accessing their own local directory. This becomes especially useful when all the query results that need to be sent back to the leader, can be put directly in a file in the shared directory, conveniently available for the leader to read from its own local directory. Using NFS only requires the workers node to send short message back to the leader stating whether the results have been added to the shared file.

C. Integrating X-Search

Finally, to finish off the distributed information retrieval middleware, the major and final step was to integrate X-Search. X-Search is built on the Borus library. In our implementation, all of the files are stored in a hash table. Whenever a worker receives a message from the leader to do the indexing, all the files from the hash table get added to a queue. Then, a new file reader driver is created for each file where the reader driver reads blocks of data from the files, The blocks of data then get tokenized and the tokens get indexed, and once every block of every file in the hash table finish the tokenization and indexing process, a message is sent to the leader letting it know that the indexing was successful. In addition, whenever there is a new query request from the leader, the indexed tokens are used for comparison with the search term and if they match, a reverse lookup is performed to get the filename that contains the search term to send it back to the leader either via google protocol buffer or network file system. In our implementation, we were only able to search for one term at a time and get only one result containing a file from each worker (if the result was found). This is something that we had to put off for future work which we will discuss later in the paper. Fig 2 displays

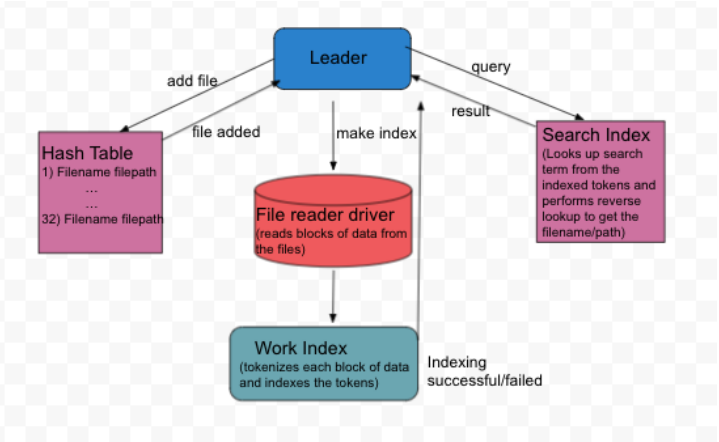

Fig. 2. Inside worker node

the workflow of search and indexing and how instructions are sent and received to each worker. It is important to keep in mind that the diagram represents communication with only one worker and in our implementation, we had connections with multiple workers working the same way.

IV. Experimental Analysis

Once we had our basic structure, we conducted four different experiments to measure the performance of the middleware. All our test was run on torus node in Mystic, which is a testbed funded by NSF. We ran our tests on 5 torus nodes, one that was designated as a leader and the rest four, as workers. As previously stated, the leader was the moderator, and it was also responsible for handling connections from the worker nodes.

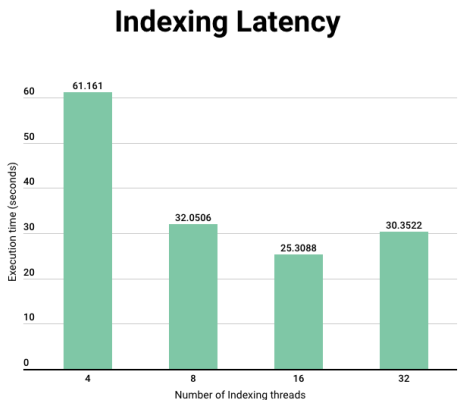

Fig. 3. Indexing latency(variation in number threads)

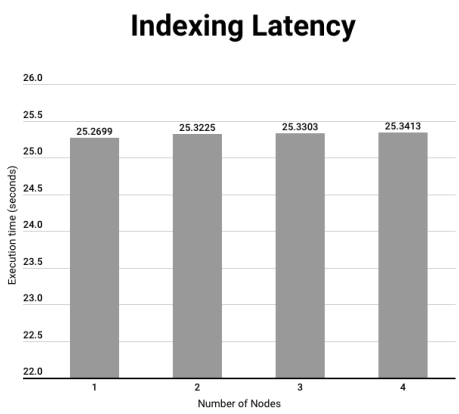

Fig. 4. Indexing latency(variation in number of nodes)

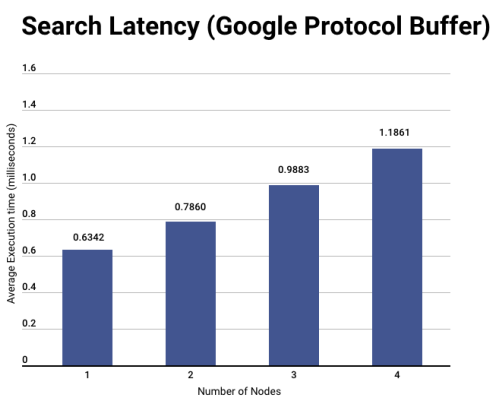

Fig. 5. Search Latency(Google Protocol Buffer)

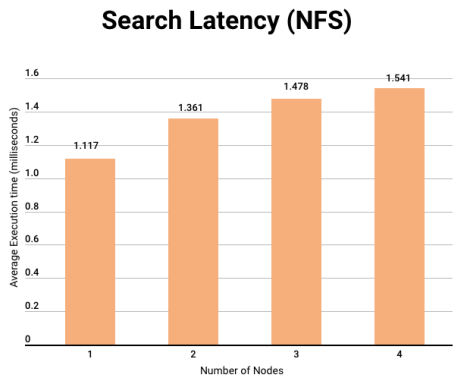

Fig. 6. Search Latency(NFS)

In our implementation, the leader was a server, and the workers were clients. The architecture of the machine we ran our test on was x86_64. Each of the nodes had 64 GB of RAM, 8 cores per socket, and 2 threads per core. For each of our experiments, we used a cluster of 256 MB of text files. We had 32 of those files in each worker node resulting in us using a total of 32 GB of files, 8 GB per node. We then partitioned our NVME drive on each of the worker nodes into four partitions and mounted each partition to a directory containing the text files.

For our first experiment, we wanted to find a number of indexing thread that would take the least time to index. We ran our program on just 1 worker node and figure 3 displays the experiment result. As we increase the number of threads, the execution time decreases for the most part, but after we exceed 16 indexing threads, the performance no longer gets better, on the contrary, we found that it takes more time to do indexing with 32 threads in comparison to 16 threads. This might be because of the overhead of creating threads itself which outweighed the compute time. Next, we conducted our experiments with variations in the number of nodes. We wanted to see how the increase in the number of nodes affects indexing and search performance. In our second experiment, we measured the time it takes for a leader to send an indexing instruction to the workers till the time it takes for the workers to do the indexing and send the result back to the leader. We found that, although there is not a substantial increase in latency, the execution time does increase with an increase in the number of worker nodes. The results are displayed in figure 4.

With the same setup, we then conducted two more experiments to measure the search performance. The only difference between the latter two experiments is that we used google protocol buffer to send the query result back to the leader in one and for the other experiment we used a network file system, NFS. We queried for 128 terms in each variation of the number of nodes in which there was a match for only half of the terms we queried for. We then took the average of the time it took to get the query result back for each of

our queries and figure 5 and 6 represent the search latency accordingly for google protocol buffer and NFS. We found that sending messages using google protocol buffer resulted in better performance in comparison to using a network file system. When sending packages using google protocol buffer, the data is compressed into binary which takes a lot less space and bandwidth. With NFS, even though the results are available for the leader to read from its own local directory, a message still needs to be sent by the worker node when a result is added, and because of that additional step, it is possible that the execution time took longer. Nevertheless, being able to search from 32 GB of files in a matter of few milliseconds where traditional tools have search latencies in the order of tens of seconds, shows a very promising result and could be transformative in information retrieval in a distributed file system [7].

V. Related work

Elastic search is a search engine based on Apache Lucene Library. In elastic search, nodes run one instance per machine, and they communicate with each other via network calls to share the responsibility of reading and writing data. A master node organizes the entire cluster. Documents are JSON objects that comprise the results that Elasticsearch is searching for, and requests are routed through nodes which the nodes then merge results from shards (Lucene indices) together to create a search result [6].

The downside of elastic search is that it was designed to be operated as a stand-alone cluster, It is not optimized for the types of data or queries commonly seen on a parallel file system [7].

Acknowledgment

Experiments for the project were conducted using the Mystic testbed funded by the National Science Foundation.

Conclusion and Future work

With the advancement in the computer system, the demand for a better information retrieval system suitable for parallel and distributed systems only gets bigger. X-Search facilitates the information retrieval in the existing parallel and distributed filesystem as a scalable solution. In creating the middleware, there still is a space for components which if added, can make the data retrieval process even faster and better. Instead of using a single-threaded non-blocking IO for communication, implementation of multi-threaded non-blocking can provide higher scalability and better performance. Future work could also involve a structure suitable for querying multiple terms and getting more than one result from a node for the query. Scaling up the experiments to more nodes can also be expected in the future.

References

[1] Maresca, P. (2016, March 6). Scalable i/o: Events- vs multithreading-based. TheTechSolo. https://thetechsolo.wordpress.com/2016/02/29/scalable-io-events-vs-multithreading-based/.

[2] 6.2. the select() API call. The select() API call. (n.d.). https://www.scottklement.com/rpg/socktut/selectapi.html.

[3] Google. (n.d.). Protocol buffers — google developers. Google. https://developers.google.com/protocol-buffers.

[4] Serialiazing your data with Protobuf. Conan C/C++ Package Manager Official Blog. (2019, March 6). https://blog.conan.io/2019/03/06/Serializing-your-data-with-Protobuf.html.

[5] Kurian, A. M. (2021, July 26). Understanding protocol buffers. Medium. https://betterprogramming.pub/understanding-protocol-buffers-43c5bced0d47.

[6] Lin, A. (2016, March 2). Elasticsearch architectural overview. Medium. https://buildingvts.com/elasticsearch-architectural-overview-a35d3910e515.

[7] A. Orhean, K. Chard and I. Raicu, ”XSearch: Distributed Information Retrieval in Large-Scale Storage Systems.”